

ChordStream

A key-relative, time-anchored chord token stream for learning cadence

PsiPi

September 2026

This article is the reference description of ChordStream, a chord representation developed alongside PsiPi/captioned-ChordStream and related datasets. It is written so the representation can be re-implemented from this page alone if the original code is lost.

Abstract

ChordStream is a compact symbolic representation of the harmony of a piece of music: one 64-bit token per chord change or silence, carrying the key, the chord's degree relative to that key, its pitch-class content, its bass, and its absolute bar, onset, bar length and duration in musical time. It is deterministic, invariant to tempo, and self-describing under windowing: any contiguous slice of tokens states its own bar, key and bar length, so it knows where its bar lines fall and how much time it covers without reference to anything outside it. We give the full extraction procedure from MIDI or from REMI token streams, including an exact reconstruction of the REMI clock that maps any window of a training stream to the chords sounding in it without decoding to notes. The extractor is evaluated against human harmonic annotation on a pop corpus and a classical one: on pop it is level with a published rule-based chord recogniser and better on bass, and it sits well behind learned recognisers, most of all on classical music. It is intended as conditioning for models learning cadence. No model trained with ChordStream is reported here.

1 Motivation

In event-based symbolic encodings such as REMI, a cadence is distributed across many pitch, position and duration tokens, and a model must infer harmonic function from them. A representation that states the harmony directly, relative to the key and anchored in time, lets a model observe a V-i approach and arrival as two tokens, identically in all 24 keys.

2 Token

Each token is two 32-bit words, packed as $a \mid (b \ll 32)$ in 64 bits.

word	field	bits	meaning
A	tonic	4	key tonic pitch class 0–11
	mode	1	0 major, 1 minor
	degree	4	root relative to tonic 0–11; 14 = solo line; 15 = silence; 12 and 13 unassigned and never emitted
	mask	11	bit i-1 set iff the interval i semitones above the root is present
	bass	4	bass pitch class relative to the root; 0 also means no bass (see below)
	barlen	6	length of this token’s bar in sixteenths, stored minus 1 (1–64)
	reserved	2	zero
B	bar	16	absolute bar from the piece start (0–65,535)
	onset	6	sixteenths after the bar’s downbeat (0–63)
	dur	8	length in sixteenths (1–255)
	cont	1	continuation of the previous token (a chord that began earlier)
	reserved	1	zero: the token’s top bit

Field order within each word, least significant bit first: `a = tonic | mode<<4 | degree<<5 | mask<<9 | bass<<20 | (barlen-1)<<24` and `b = bar | onset<<16 | dur<<22 | cont<<30`.

The token’s top bit is always zero, so a token is equally safe held as `uint64` or `int64`. This is deliberate: `numpy` and `Arrow` both mishandle `uint64` values above $2^{63} - 1$, `numpy` by promoting them to `float64` when they meet a signed type, `Arrow` by refusing them outright.

Properties. For `degree` in 0–11 the absolute root is `(tonic + degree) mod 12`. The formula does not apply to the two reserved degrees: when `degree` is 14 (solo) or 15 (silence) there is no root, and `mask` and `bass` are both zero. All fields except the tonic are relative, so transposition changes 4 bits and a mislabelled key changes a chord’s name, never its identity. Spelling is not represented (MIDI has none). The mask represents any pitch-class set above the root. Time is in sixteenths derived from ticks; tempo is never read.

`bass` carries 0 for two different things: a chord voiced with its root in the bass, and a step where no note met the threshold to count as bass at all. A reader cannot tell them apart. The field has four unused values (12 to 15) that could carry an explicit no-bass marker, and we have not spent one, because no consumer has needed the distinction; it is recorded here as a known ambiguity rather than an oversight.

`barlen` is the length of the bar the token sits in. It is the effective bar rather than the time signature, because the two differ: bars longer than 64 sixteenths are split into sub-bars (below), so a 166/4 piece carries 8 here and a 17/4 piece carries 36 then 32. A signature field could not serve the same purpose, being wrong about the bar it is attached to.

What `barlen` is for is bar-line arithmetic inside a window: where the next downbeat falls, how much of a bar a chord occupies, whether a chord crosses a bar line. A slice could not do any of that before without an external time-signature table. Elapsed time between tokens never needed it, since silence is emitted as its own token and only trailing silence is dropped, so tokens abut and the gap between any two is the sum of the intervening durations.

It does **not** give absolute position from the start of the piece. That is `sum(length(i) for i in 0..bar-1) + onset`, and a token knows only its own bar’s length. `bar * barlen + onset` is correct only for a piece of uniform bars, and wrong wherever a pickup, a metre change or a split long bar precedes the token: after a 1/4 pickup, a token at bar 1 with `barlen` 16 computes 16 where the true position is 4. Absolute position is therefore not a property of the token. Where it is needed, the extracted sidecars carry an exact per-token `start16` alongside the stream.

What a window does state for itself is its bar number, its key, and the length of the bars it covers: a window whose first token has bar > 0 or onset > 0 is known to begin mid-piece, and it can measure its own extent and place its own bar lines without looking outside the slice.

Two bits of word A are held in reserve, and word B's top bit must stay zero for the token to remain safe as a signed integer, so those two are the only headroom the format has. We have deliberately not spent them on the metre. Bar length identifies five of the nine metres our REMI tokeniser emits ($3/8$, $1/4$, $2/4$, $4/4$ and $5/4$, at 6, 4, 8, 16 and 20 sixteenths) and leaves four ambiguous in two symmetric pairs: a 12-sixteenth bar is $6/8$ or $3/4$, a 24-sixteenth bar is $12/8$ or $6/4$, each pair differing in chord step (6 against 4) rather than in length. A consumer that needs the step can take it from a sidecar column, which costs one pass over a corpus and no change to the token or to any reader, so the bits stay free for a flag that has to live in the token itself.

3 Extraction

Input is a list of pitched notes (drums excluded) with onsets and offsets in sixteenths, each tagged with a voice, and a list of time signatures. For MIDI files we use symusic, whose tracks split a MIDI track at channel or program changes; a voice is such a track.

The order of the notes matters, and must be reproduced. Notes are listed voice by voice in symusic's track order, and within a voice by onset, with notes of equal onset kept in the order their note-on events appear in the file (not note-off order, and not by pitch). Two rules below walk notes in that order and compare each with the one before it where onsets tie, the monophony count of the melody-voice rule and solo rule B, so a reader that lists the same notes in another order can reach different results.

Time is not quantised on input. A note's onset and offset are its ticks multiplied by 4 over the file's ticks per quarter note, kept as real numbers, so a note at tick 119 of 120 per quarter lies at 3.9667 sixteenths and overlaps steps by that real amount; notes of zero length are dropped. Every overlap, histogram and threshold below is computed in that continuous time, and comparisons of positions allow a tolerance of $1e-6$ sixteenths. Integers appear only at emission (below).

The steps run in this order: bars and chord steps from the metre; a chord fitted to every step, with pedal points and solo lines resolved; the key of every bar; merging; emission. Chord fitting never consults the key, and the key is estimated after the chords so that it can use them. That use is not circular: chord extraction is key-independent, and key inference consumes the resulting roots and basses but never feeds back into fitting. The paragraphs below describe each step, not in that order.

Metre. Bars follow the time signatures, laid one whole bar at a time from the start of the piece: at each bar line the signature in force is the last one placed at or before that bar line, and the bar takes that signature's full length. A signature placed inside a bar therefore does not cut the bar short; it takes effect at the next bar line. Of several signatures at the same position the last one listed wins, and where the first signature starts after 0, $4/4$ applies before it. A metre written as a single unreduced beat (numerator 1, denominator 8 or finer: $1/8$, $1/16$, $1/32$) and persisting for more than two bars is a placeholder and is treated as $4/4$; short pickup bars are kept. The test is on the written signature, not on bar length: $3/8$ is 6 sixteenths and $1/4$ is 4, both shorter than a $2/4$ bar and both legitimate, and rewriting either to $4/4$ would advance the bar accumulator by 16 instead of 6 or 4 and desynchronise every later bar line. No such placeholder signature exists among

the nine metres the REMI tokeniser supports, so the rule is inert on token-sourced extraction and active only on MIDI.

The chord step is the metre’s beat (three denominator units in compound metres: numerator divisible by 3 and > 3 , denominator ≥ 8) grouped: the smallest whole multiple of the beat that is at least a quarter note and divides the bar, or the whole bar if no multiple does, and never longer than the bar itself. Only multiples of the beat are candidates, which is why 5/8 steps by 10 sixteenths: its beat is 2, and 4, 6 and 8 do not divide 10, while 5 is not a multiple of the beat. An irregular metre is therefore fitted a bar at a time; its internal grouping, 3+2 or 2+3, is not modelled. Where the bar is shorter than a quarter note, as in a 1/16 or 3/16 pickup, the step is the whole bar, since no step of 4 sixteenths or more can divide a bar of 1, 2 or 3. Grouping gives 4 sixteenths for 4/4, 3/4 and 7/4; 8 for 2/2; 6 for 6/8, 12/8 and 12/16; 4 for 16/16; 10 for 5/8. Bars longer than 64 sixteenths, which the 6-bit onset field cannot address, are split by numerator. Let n/d be the bar and k the fewest parts that could fit, the ceiling of its length in sixteenths divided by 64. First, for each whole number m from k up to $n/2$ in turn, if m divides n and n/m units of $1/d$ fit in 64 sixteenths, the bar becomes m equal sub-bars of $(n/m)/d$; the cap at $n/2$ keeps every sub-bar at least two units long. Failing that, n is divided into k near-equal parts, increasing k until the largest part fits: with $n = qk + r$, the bar becomes r sub-bars of $(q+1)/d$ followed by $k-r$ of q/d , the longer ones first. So 166/4 becomes 83 bars of 2/4 and 17/4 becomes 9/4 then 8/4. Each sub-bar is an ordinary bar from then on: it has its own start, its own **barlen** and its own chord step computed from its own numerator, bar numbers count sub-bars, and a chord step never crosses a sub-bar boundary. Ordinary long metres (9/4, 11/4, 12/4, 15/4, 7/2) fit whole.

The REMI tokeniser used in our corpora, a miditok 3.0.6 configuration read with symusic 0.6.0 (the versions recorded in its `tokenizer.json`), supports nine metres (3/8, 6/8, 12/8, 1/4, 2/4, 3/4, 4/4, 5/4, 6/4); 7/4, 9/4, 5/8 and 2/2 are not among them, and miditok writes such pieces as 4/4. Our training pipeline has been forced to adjust for this in its time-signature conditioning field, refolding unsupported metres where the bar can be written honestly (2/2 as 4/4, 9/4 as three bars of 3/4) and naming prime-odd metres such as 7/4 in the caption instead. This does not affect ChordStream, which takes its metre from the notes it is given: the file’s own metre when extracting from MIDI, the tokeniser’s when extracting from REMI tokens.

Key. Per bar, a duration-weighted pitch-class histogram. For bar i of a piece of N bars the histograms of bars lo to $lo+7$ are summed, with $lo = \max(0, \min(i-4, N-8))$: bars $i-4$ to $i+3$ in the body of the piece, bars 0 to 7 for every i up to 4, and bars $N-8$ to $N-1$ for every i from $N-4$. Where N is under 8, lo is 0 and the window is simply every bar of the piece, summed with no padding; Pearson correlation does not depend on the histogram’s scale, so nothing is rescaled. The summed histogram is correlated (Pearson) with the 24 rotations of the Aarden-Essen major and minor profiles, and 0.05 is added to the correlation of the whole-piece key before anything else is decided: before the best key in the window is chosen and before the relative tie test below, which compares the scores with that bonus included. That whole-piece key is the same calculation over one histogram covering every bar, correlated against the same 24 rotations and resolved by the same relative-major rule. When the best key and its relative major/minor differ by at most 0.03, the tie is settled by tonic evidence taken from the fitted chords over the same 8 bars. Each pitch class earns the sixteenths in which it is the step’s bass, plus the sixteenths in which it is a chord’s root, so a root-position tonic counts twice and an inversion or a pedal once; chord quality is never used, so a passage of power chords with no third anywhere still supports its tonic.

It also earns closure, defined over root runs. Taking the chord steps of the piece in order and

skipping silence and solo steps entirely (they neither end a run nor separate two), a root run is a maximal sequence of consecutive chord steps with the same root; its length is the sum of their lengths and its bar is the bar of its first step. An arrival is every run after the first, arriving from the run before it. If the earlier run’s root lies 5, 7 or 10 semitones above the arrival’s root (a fourth, a fifth or a whole tone above), the arrival’s root earns the arrival’s length, credited to the arrival’s bar; if that interval is 10 and the run before the earlier one has a root 8 semitones above the arrival (bVI then bVII then I), it earns the length a second time. Evidence of both kinds is stored per bar and summed over exactly the bars of the 8-bar window the profile was scored on, or over the whole piece for the whole-piece key, so an arrival counts in a window when its first step lies there. These rules are identical for a major and a minor candidate. Section 5 measures what this recovers, which is little.

The key of one window is therefore decided in this order. (1) Correlate the window’s histogram with the 24 keys and add 0.05 to the whole-piece key. (2) Take the highest score; on an exact tie the lowest index wins, majors being indices 0–11 on C to B and minors 12–23. (3) If that key and its relative major or minor differ by more than 0.03, keep it. (4) Otherwise compare the tonic evidence of the two candidates over the window, and if they differ by more than $1e-6$ the larger wins. (5) Otherwise choose minor iff the minor key’s raised leading tone has more weight than its subtonic in the window’s histogram, and major if not, a draw included. A window with no sounding notes takes the previous bar’s result, or the whole-piece key at bar 0.

Hysteresis then turns these per-bar results into keys. Bar 0 takes its own result outright as the current key. At each later bar: if the result equals the current key, any pending candidate is discarded; if it equals the pending candidate, that candidate’s count rises by one; otherwise it becomes the pending candidate with a count of 1. When the count reaches 4, the candidate becomes the current key, and the change is applied retroactively: those four bars, the first three already labelled with the old key, all take the new one. Nothing else accumulates. A candidate’s count is not kept when another key interrupts it, including a return to the current key, so a change needs four consecutive bars won by the same key, and two competitors alternating never displace the current key. The key is always set.

Aarden-Essen profiles used (pitch classes from the tonic):

```
major: 17.7661 0.145624 14.9265 0.160186 19.8049 11.3587 0.291248 22.062 0.145624 8.15494 0.232998 4.95122
minor: 18.2648 0.737619 14.0499 16.8599 0.702494 14.4362 0.702494 18.6161 4.56621 1.93186 7.37619 1.75623
```

Mask. The mask is the winning template’s interval set, not the sounding pitch content: for maj7 at any root it is intervals 4, 7 and 11, so bits 3, 6 and 10. Nothing about the step’s actual pitch classes reaches the mask beyond its effect on which template won, so a passing tone never sets a bit and no weight or register filter applies to it. The field is 11 bits rather than the 4 a template index would need so that a future extractor can write an arbitrary pitch-class set above the root without a format change; the extractor described here only ever writes one of the template patterns, and by default one of fourteen (add9 and madd9 are emitted as their triads, below).

Melody voices. A voice is melodic if it is at least 85% monophonic, its mean pitch exceeds the piece median by more than 2 semitones, and its mean pitch is at least 60. All three measures count notes, not durations. With a voice’s notes ordered by onset, monophony is 1 minus the number of notes that begin more than half a sixteenth before the previous note ends, divided by the number of notes less one. The mean is the unweighted mean of the voice’s note pitches, and the piece median

the unweighted median of every note’s pitch in every voice, so a walking bass of many short notes pulls the median down more than a few sustained notes would. The rule is applied only when the piece has more than one voice. If no voice qualifies, the top note at each onset is marked melodic when it is at least 60 and at least 5 semitones above every other sounding note. Onsets are grouped after rounding to two decimal places of a sixteenth, exactly as Python’s `round(x, 2)` rounds the binary value (so an onset of 1.125 groups at 1.12); within a group the top note is the first of the highest pitch, and “every other sounding note” means the rest of its group together with every note from an earlier group whose offset lies after the group’s rounded onset.

Weights. A note contributes its overlap with a step, multiplied by 0.25 if melodic; otherwise by 1.5 below pitch 60, by 0.6 at or above pitch 72, and by 1.0 in the midrange between them (pitches 60 to 71). The step’s bass is the lowest pitch overlapping at least a quarter of the step. A step can sound and still have no bass under that rule, every note in it being too short, as in a step of staccato sixteenths: the step then has no bass, the root bonus below does not apply, and the token’s `bass` field is zero, the same value a chord voiced with its root in the bass carries.

Chord fit. Sixteen templates (intervals above the root): maj (0 4 7), m (0 3 7), 7 (0 4 7 10), m7 (0 3 7 10), maj7 (0 4 7 11), 5 (0 7), sus4 (0 5 7), sus2 (0 2 7), dim (0 3 6), aug (0 4 8), m7b5 (0 3 6 10), dim7 (0 3 6 9), 6 (0 4 7 9), m6 (0 3 7 9), add9 (0 2 4 7), madd9 (0 2 3 7). With `h` the step’s 12-bin weighted histogram (each bin the sum of its notes’ weighted overlaps with the step), $w = h / \text{sum}(h)$, so that the twelve values of `w` sum to 1; the penalties below are calibrated to that scale and are meaningless under any other normalisation. With `T` a template at root `r`:

```
s = 2 * sum(w[p] for p in T) - 1
    - 0.12 * count(p in T with w[p] < 0.03)
    - 0.06 * max(0, |T| - 3)
    + 0.08 * [has_bass and bass % 12 == r]
```

`has_bass` must be tested separately and not folded into a sentinel value. A step with no bass carries 0 in the token’s `bass` field, and an unguarded `bass % 12 == r` would then award the bonus to root C in every such step, biasing staccato passages towards C.

The maximum over the 192 candidates wins; ties within 1e-9 go to the first candidate ordered by root pitch class 0–11, then by template index 0–15 in the order listed above. A step is silence when `sum(h)` is below 1e-6, which includes a step with no sounding notes and one whose notes overlap it by a negligible amount.

By default a winning `add9` is emitted as its major triad and a winning `madd9` as its minor triad, root unchanged; the refit after merging applies the same rule. Most `add9` readings are a melody note a ninth above a triad rather than an added-ninth chord, and against human labels (section 5) emitting the triad raised quality accuracy by 14.9 points. The option to keep them remains.

Pedal points. Where the step bass holds the same pitch for at least 32 sixteenths, that pitch is removed, the upper voices are refitted at half-bar resolution, and the pedal is restored as the step’s bass. The step’s absolute bass pitch is untouched by this: a step keeps the absolute pitch it found throughout, and the conversion to an interval above a root happens only where a chord is named, so a refit or a merge that comes later still has a real MIDI pitch to compare. That is what makes the absolute comparison in the merge below well defined inside a pedal span, where every step’s bass is the pedal. The half-bar grouping is by label, not by cutting: each group is the run of steps

beginning in the same half of the same bar, so no step is ever divided and an odd bar simply groups unevenly. A 3/16 bar of one step is one group.

Solo lines. A step with no sounding notes is silence and is never solo, whichever rule is applied. Subject to that, a step becomes solo (degree 14) if (A) melody voices were found by the voice rule, non-melodic notes are at least 10% of all notes in the piece (a count of notes, not of duration), and non-melodic notes sound for less than half a sixteenth in the step; or (B) the step's notes, taken in order, are monophonic and contain fewer than three pitch classes. Monophonic here has the same tolerance as the melody-voice rule: with the notes sounding in the step in onset order, a note counts as polyphony only when it begins more than half a sixteenth before the note before it ends, so legato overlaps shorter than that are allowed. Rule B spares monophonic steps of three or more pitch classes, which may be arpeggios.

The sounding guard is not decoration, and it has to be stated once for both rules rather than attached to either. An empty step satisfies B vacuously, no two of its nought notes overlapping and nought being fewer than three. It satisfies A just as vacuously by the other route: the first two conditions are properties of the piece and hold throughout, and nought non-melodic notes sound for less than half a sixteenth. Without the guard every silence in a piece with a detected melody voice would be labelled a solo line.

Clean-up. By default nothing is absorbed. As an option, a single-step chord between two identical chords is absorbed, unless the step itself is on degree 7 or 11, or its two neighbours are. It is off because, against human labels, it made no measurable difference to accuracy (every chord metric within 0.1 point) while erasing real harmony: 69% of absorptions change nothing, the root merge collapsing them anyway, and among the rest the most common erasure is I-IV-I. The rest of this paragraph describes the option. The rule fires only when the neighbours are identical, which is the shape of a cadence, so without those exceptions it erases exactly what this representation exists to expose: the dominant of an i-V-i, or the resolving tonic of a V-I-V dominant prolongation. Measured on 600 documents, the exceptions keep 30.6% of all would-be absorptions, 3,368 of 11,018: 2,429 dominants on degree 7 and 213 on degree 11 as the middle step, and 726 further steps whose neighbours are dominant, of which 274 are a tonic inside a V-I-V.

The exception is on dominant function, not on the tonic. Protecting degree 0 outright would be wrong: 82% of absorbed tonics sit between tonic neighbours and differ only in quality or bass (C, Cmaj7, C), which is the noise the rule exists to remove and which the root merge that follows would collapse in any case. Degree 5 is absorbable under the option, which is one reason it is not the default: a single-beat IV between two tonics is often a real plagal event.

Absorption applies only where the step and both neighbours are chords. A single step of silence between two identical chords is kept, and so is a single chord between two steps of silence: a stab in the middle of a rest is the music rather than a blip to smooth away, and it occurs in 0.029% of steps.

Merging and refitting. Consecutive steps in the same key are merged: steps on degrees 0–11 when they share a root, and silence and solo steps when they share that kind, since neither has a root to compare. A merged chord span is then refitted exactly as a single step is, with three specifics. The weighted histogram is re-summed over the whole merged span, using the same per-step histograms the fit used (so a pedal span uses its pedal-removed histograms). The template is

the one maximising the same score with the root held fixed at the merged root, so only the quality can change. The bass is the lowest among those merged steps that had a bass at all, compared as absolute MIDI pitches and only then converted to an interval above the merged root; comparing the stored root-relative values would be meaningless, since a low B under a C root stores as 11 and a higher E stores as 4. Steps with no bass take no part in the comparison rather than contributing a zero, and if no merged step had a bass the merged span has none. Each bass that does count already met the quarter-of-its-step threshold and none is re-thresholded against the merged duration, so a four-bar span of bar-long bass notes keeps its bass rather than losing it for covering under a quarter of the span. Merged silence and solo spans have nothing to refit.

A merged chord span’s refit can find nothing to fit, and not only in theory: inside a pedal span the refit reads the pedal-removed histograms, so a span where only the pedal sounded sums below the silence threshold. The span is then not turned into silence. It keeps the chord of its first step unchanged, root, template and bass, and is emitted with that chord over its whole length. A merge never joins steps from different keys or of different kinds, so a refit is only ever asked about one root in one key.

Emission. Integer fields are computed at this point and nowhere earlier: **onset** is the token’s start minus its bar’s start, rounded to the nearest integer and capped at 63; **dur** is the token’s span rounded, at least 1; **barlen** is the bar’s length rounded and clamped to 1–64. Rounding is to the nearest integer with exact halves going to the even neighbour (2.5 becomes 2, 3.5 becomes 4), as Python’s **round** does; an implementation that rounds halves upwards will differ on every exact half-sixteenth. Any span held across a change of bar length is split at the change, a chord, a solo line and a silence alike, because a token carries one **barlen** and would otherwise describe its later bars with the wrong bar length, breaking the only thing **barlen** promises; this affects 0.21% of tokens. Merging and refitting come first and ignore bar lines of every kind: steps merge across bar boundaries, bar-length changes and sub-bar boundaries alike, and the refit sees the whole merged span. The split at a bar-length change is applied only afterwards, to the refitted span, so every segment carries the same degree, mask and bass, and only **bar**, **onset**, **dur**, **barlen** and **cont** differ between them. A segment is cut at the start of the first later bar whose length differs from the length of the segment’s own starting bar, so a span crossing 4/4, 3/4 and back to 4/4 becomes three segments, one per run of equal-length bars. Tokens longer than 255 sixteenths are likewise split into consecutive tokens. In both cases the second and later segments carry the continuation flag and the first does not, so a stream reader meeting a token with **cont** set knows the chord began before it. The flag marks only these two splits, which the extractor emits; ordinary bar lines are not split, so **cont** is not a general statement that a token begins its chord, and a consumer cutting a stream at an arbitrary time must set it themselves (section 4). The first segment’s flag is never inherited from what precedes it. Trailing silence is dropped; bars beyond 65,535 are not emitted.

4 Reading REMI token streams

Training corpora are often stored as REMI tokens with byte-pair merges (miditok). A document is decoded once to a score and extracted as above. To align a window of the token stream with time without decoding, each merged id is expanded to its base tokens and miditok’s REMI clock is replayed:

- a Bar token advances one bar;

- a Rest token advances by its own encoded duration, from the later of the current time and the latest note end, and recomputes the bar index from the last time-signature change. The latest note end is one running maximum over every note in the stream, whatever its voice: miditok's REMI writes each note as consecutive Pitch, Velocity and Duration base tokens (Pitch and Duration where velocities are disabled), so a Pitch's duration is the Duration token two base tokens after it (one without velocities), and the maximum is raised to that note's onset plus its duration. Polyphony needs no per-voice tracking, because only the latest end matters to a Rest. A Tempo token does not advance the clock; for consistency with the decoded stream, the running latest note end is updated with the current time when one is read;
- a Position token sets the offset within the bar;
- a TimeSig token changes the bar length and re-anchors bar counting.

Counting Bar tokens alone is incorrect, since rests may cross bars without Bar tokens; on 29 sampled documents it matched the decoded bar count 3 times. The replayed clock placed 146,109 of 146,109 checked note-bearing ids exactly on a decoded onset (30 documents), at 27 ms per document.

Windowing. Let $t(i)$ be the clock after the replay has consumed token i . A window's span is taken from the window's own tokens: it begins at $t(f)$, where f is the first position in $[s, e)$ carrying a pitch, and ends at the latest $t(i)$ over positions i in $[s, e)$ that carry a pitch or a rest. A Pitch token does not advance the clock, so t at such a position is that note's onset. A Rest is silence the window consumes and so extends it; a trailing Bar or Position does not, because it only sets up a note the following window owns, and letting it extend the window would pull in the next bar's chord, which is the same mistake at the other end. Rests after the last note occur in 1.3% of 512-token windows, worth a median of 2 sixteenths. A window containing no pitch at all has no f , and what it covers depends on whether it holds rests. If it does, it covers their silence: with r the stream index of the first rest in $[s, e)$ and r' that of the last, the span is $[t(r-1), t(r'))$, open at the top, $t(r-1)$ being where that first rest began and $t(-1) = 0$ at the start of a document as before. Reading the clock after the window's last token instead would land exactly where the next sound begins and hand a window of pure silence the following bar's chord, which is the rest-end trap again in its purest form. If the window holds no rest either, only bars and time signatures, it covers the single instant $[T, T]$ with $T = t(e-1)$, closed, matching whatever token spans that instant. Pitchless windows are 0.04% of 512-token windows and 0.96% at 32 tokens.

Whether the upper bound is closed depends on what produced it, and this is the one place in the format where a bound's openness is derived rather than chosen. When the end is an onset, the bound is closed: a chord beginning exactly on the window's last onset sounds under that note and belongs to the window. When the end is a rest, the bound is open: a rest runs up to where the next sound begins, so a chord starting exactly there is the next window's, caused by notes this one does not contain. Ends of the second kind are 1.2% of windows. For a window containing pitch the start bound is always $t(f)$, and always closed.

The start is the first onset, and silence before it belongs to the previous window. A window that opened with a rest before its first note would, on that convention, not cover the rest. We measured this on 2,721 windows of 512 tokens and found no case at all: a Bar token is followed immediately by its Position and first Pitch at the same clock, so the window's first pitch already sits at the earliest time the window asserts. Taking the start from the window's first token instead would not have recovered a rest either, since the clock at a Rest is the rest's end, and it would have imported a stale onset in 11% of windows, those beginning inside a note whose Pitch sits in the previous window.

Except where a trailing rest extends it, the span stops at the last onset rather than the last release. A chord change falling after every onset in the window is driven by notes the window does not contain, so including it would hand the model harmony it cannot see the cause of. A rest is the one thing that may push the end past an onset, and it is not an exception to that principle but an application of it: the rest is in the window, and its own silence is the window’s. The cost is that a long final note’s later harmony is not described, which we accept: the chord sounding at its onset is returned, and the rest belongs to the next window.

Neither bound may be taken from outside the slice. Using $t(s-1)$, the clock before the window’s first token, is wrong whenever a window opens on a structural token: a window beginning at `Bar_N` reads the clock before that token advanced it, so it claims the whole of bar N-1 as covered and collects that bar’s chord. Measured on 150 documents at a 512-token window, that moved 20.7% of window starts, by up to 96 sixteenths, and changed the chord set of 6.1% of windows. The upper bound has the mirror-image history: it was open regardless of what produced it, which dropped a chord beginning exactly on the window’s last onset whenever a chord change coincided with the end of a window, an alignment that is anything but rare.

The window’s chords are the tokens overlapping that interval, closed at the upper bound when the end is an onset and open when it is a rest, with a flag when the first began before the window.

Slicing an extracted stream. Cutting an already-extracted token stream at token boundaries is plain slicing, and every token in the slice is complete and self-describing. Cutting at an arbitrary *time* is not: a chord held across a bar line is one token, so a consumer who wants a window to begin at bar 5 when the sounding chord began at bar 4 must split that token itself, reducing `dur` by the elapsed part, moving `bar` and `onset` to the window’s start, setting `barlen` to that bar’s length, which differs from the original token’s wherever the split crosses a metre change, and setting `cont` to 1. The extractor sets `cont` only for splits it emits, at a bar-length change or the 255-sixteenth limit, so a token-boundary slice never needs it and a time-boundary slice cannot rely on it.

5 Observations and evaluation

Density and speed. Over 397 distinct documents of the phase-B corpus the extractor yields a median of 1.57 tokens per bar (quartiles 1.18 to 2.06), so 64 bars is about 100 tokens and 800 bytes. The figure is set by how often the harmony changes rather than by how many notes there are: the densest quarter of those documents, at 39 or more notes per bar, yields 1.65. Extraction takes a median 33 ms per document on one core once the notes are decoded, and 98 ms per piece from a MIDI file across 949 unusually long pieces (median 227 bars).

Evaluation against human annotation. The extractor was scored against two human-labelled test sets, using the protocol of BACHI, the paper that introduced the first of them, so that results sit beside published systems. Chords are scored on four elements, root, quality, bass and the full chord, at two frames per beat, averaged per piece and then over pieces. Both sides are named by the label set’s own decoder: ChordStream’s chord, whose pitch classes are exactly its root and mask, is passed through the same function that named the labels, so a C6 reads Am7 exactly as a label with those notes would. Keys are scored by the MIREX weighted score (1 for the correct key, 0.5 a fifth, 0.3 the relative, 0.2 the parallel), with relative major/minor confusions also counted on their own, since the weighted score partly credits exactly the error this representation most needs

to avoid.

The evaluation code was checked against an independent system. Run through it on the original scores, BACHI’s released classical model scores 78.2, 79.1, 77.5 and 68.5 against its published 77.8, 79.0, 77.0 and 68.1, and its pop model lands within 1.6 points of its published figures once BACHI’s own preprocessing, which moves each piece to start on beat 0, is reproduced.

POP909-CL, human-corrected chords and keys for pop piano arrangements, on BACHI’s published 91-piece test split:

	root	quality	bass	full
ChordStream	86.0	70.4	91.2	67.3
rule-based recogniser that produced the original POP909 labels (published)	85.9	69.7	85.8	65.0
BACHI, same evaluation	93.3	88.2	95.2	86.1

When in Rome and DCML, human functional analyses of classical music, on BACHI’s published 150-piece test split:

	root	quality	bass	full
ChordStream	60.9	52.5	61.4	39.1
BACHI, pop-trained model	62.7	60.1	64.8	46.0
BACHI, classical-trained model	78.2	79.1	77.5	68.5

On pop, ChordStream is level with the rule-based recogniser on root, quality and full chord and better by 5.4 points on bass. The margins other than bass are smaller than the differences between evaluators measured above, so they cannot be called either way, and the POP909-CL labels were corrected from that recogniser’s own output, which tilts borderline cases towards it. Against a learned recogniser the gap is large: 18.8 points of full chord on pop. On classical music ChordStream is much weaker than on pop, and so is a learned model trained on the wrong repertoire: BACHI’s pop model falls from 86.1 to 46.0 full chord, keeping a lead of 6.9 points over ChordStream and less than 2 on root, while its classical model leads by 29.4.

Two extraction defaults were set by these measurements. Emitting add9 and madd9 as triads raised quality accuracy from 54.2 to 69.1 and full-chord accuracy from 52.1 to 65.8 across all 907 POP909-CL pieces; the label vocabulary has no added ninth, but the gain is mostly melody notes being read as the melody they are. Turning absorption off changed no chord metric by more than 0.1.

Keys. On the POP909-CL test split exact key accuracy is 79.7% (MIREX 85.1); across all 907 pieces, 82.4%. The largest single error is the relative major or minor, 9.3% of frames, and it runs the way the leading-tone test predicts: 70% of those frames are minor read as major. The tonic evidence of section 3 recovers little of it, 0.9 points of exact accuracy on the test split and nothing measurable on classical, and the reason is that the error mostly is not a near-tie. Only 5.7% of bars where minor was read as major fall inside the 0.03 tie window; for the rest the profile correlation itself prefers the relative major, by a median of 0.168. On classical music exact key accuracy is 62.9% (MIREX 72.1), but those labels mark brief tonicisations that an 8-bar window with 4-bar hysteresis is built to ignore, so that figure understates the detector on its own terms. Correlating the profile against fitted chord tones instead of notes gained 1.7 points on pop and lost 1.4 on classical, and Krumhansl-Schmuckler profiles in place of Aarden-Essen lost 8 points on pop, almost all to fifth confusions; neither is used.

6 Limitations

The extractor is heuristic. It has been measured against human annotation on pop and classical music (section 5), but not on the electronic, rock, jazz, ambient or game music that make up much of the corpora it was built for, where no labelled symbolic data was available. On 949 long unlabelled pieces from those genres, ChordStream and BACHI’s pop model agreed on the root only 68% of the time, against 87% on pop, and genuinely different chords made up 40% of their disagreements; which of them is wrong there is unmeasured. Power chords and suspended chords remain frequent readings, and power chords have no counterpart in the label vocabulary. Only major and minor keys are modelled. Irregular metres are fitted a bar at a time: a $5/8$ or $7/8$ bar is one chord step, so harmony that changes inside such a bar is not seen.

The key detector is biased, not merely noisy, and the bias is in the profile correlation rather than in any tie-break. Pop minor music, moving through i, VI, III and VII with melodies resting on the third and fifth, fills a pitch histogram with its relative major’s tonic triad, and the Aarden-Essen correlation confidently prefers that major key. Tonic evidence from bass and closing root motion corrects near-ties but cannot reach confident errors, and fixing them would mean fitting weights or a pop-derived profile on held-out annotated data. Phrase-final evidence would help and is not used, because MIDI marks no phrases. Harmony implied by a single arpeggiated line across a bar is not recovered. Several thresholds were set by inspection of a small number of pieces. No model trained with ChordStream is evaluated here.

Worked example, the worst compared documents. One file is 1,554 notes over 2,488 sixteenths, but the file is written in $1/8$ and $7/8$ bars as well as $4/4$, $3/4$ and $1/4$, and the REMI tokeniser has neither $1/8$ nor $7/8$. A $1/8$ pickup puts every later bar line two sixteenths out of step, the $7/8$ bars add further drift, and only 4 of about 190 bar lines coincide. No constant time shift repairs it, and the disagreements are the neighbouring chords of the progression rather than unrelated ones: the same music, cut in different places. Neither REMI nor ChordStream gets the reading wrong; it is just different, and for training on a token stream the token-derived reading is the one that matches what the model sees.

Expressive performances are the case to watch. Where a file is a recorded performance rather than a quantised score, with a dense tempo map and notes played around the beat rather than on it, as in MAESTRO, bar lines are nominal and the chord steps placed on them may not line up with the harmonic rhythm a listener hears. We have not measured this: the nearest cases tested were two piano performances of 130 and 156 minutes carrying over 10,000 tempo changes each, which encoded without error but were not checked against human annotation.

7 Prior work and influences

ChordStream was conceived from the question of how best to encode harmony for a model using rotary position encodings, and evolved into this less fragile bit stream; it does not itself use rotary encodings. It overlaps with the chord encoders of Magenta (magenta-js `chords.ts`: major/minor, triad and pitch-class chord encodings used as per-step conditioning, e.g. in ImprovRNN), which we reviewed during design; relative to the pitch-class variant, ChordStream adds key-relative degree, bass, extensions and explicit timing. Key finding follows the Krumhansl-Schmuckler correlation

method with Aarden’s (2003) Essen-derived profiles. Template-based chord recognition follows Pardo and Birmingham. REMI is due to Huang and Yang (Pop Music Transformer); the token streams read here were produced with miditok, and files are parsed with symusic. The evaluation uses POP909 (Wang and colleagues) in its human-corrected edition POP909-CL, the When in Rome meta-corpus (Gotham and colleagues) and the DCML corpora, and follows the evaluation protocol, test splits and released models of BACHI.